Pointers in C – Notes

Definition of Pointers

- Pointers are variables that store **memory addresses.**
- They are used to access and manipulate data stored in memory.

Role of Pointers

- Crucial in C programming for:
  - Efficient memory management
  - Dynamic memory allocation
  - Passing arguments to functions

Basics of Pointers

Declaration of Pointers

```
int *ptr;
```

- ptr is a pointer variable that can store the address of an integer.

Pointer Initialization

```
int x = 10;
int *ptr = &x;
```

- & is the **address-of operator**
- ptr now stores the address of variable x

Understanding Memory

- Example:

```
int x = 10;
int *ptr = &x;
```

- x → value = 10, stored at some address (e.g., 2000)
- ptr → stores address of x (2000)

Dereferencing Pointers

Dereferencing Operator (*)

- Used to access the value stored at the address

Example:

```
int x = 0;
int *p;
p = &x;
*p = 5;
```

- This means:
  👉 Store value 5 in the memory location pointed by p

Complete C Programs

Program 1: Pointer Declaration, Initialization, Dereferencing

```
#include <stdio.h>

int main() {
    // Declaration of a pointer
    int *ptr;

    // Initialization of pointer
    int num = 42;
    ptr = &num;

    // Dereferencing pointer
    printf("Value of the variable 'num': %d\n", *ptr);

    return 0;
}
```

Program 2: Printing Address Stored in Pointer

```
#include <stdio.h>

int main() {
    int *ptr;

    int num = 42;
    ptr = &num;

    printf("content of ptr is %p \n", ptr);

    return 0;
}
```

Program 3: Combined Declaration + Initialization

```
#include <stdio.h>

int main() {
    int num;
    int *ptr = &num;

    printf("content of ptr is %p \n", ptr);

    return 0;
}
```

Program 4: Modifying Value Using Pointer

```
#include <stdio.h>

int main() {
    int num;
    int *ptr = &num;

*ptr = 10;

printf("value of num is %d", num);
printf("\ncontent of ptr is %p \n", ptr);
```

```
return 0;
}
```

Pointer Example (Error in Code)

Given Code:

```
#include <stdio.h>

int main() {
    // Declaration of a pointer
    char num;
    int *ptr = &num;

    *ptr = 10;
    printf("value of num is %d", num);
}
```

Problem:

- num is of type char
- ptr is of type int *
- Type mismatch → leads to incorrect behavior

Correct Code:

```
#include <stdio.h>

int main() {
    int num;
    int *ptr = &num;

    *ptr = 10;

    printf("value of num is %d", num);
    return 0;
}
```

Null Pointers

- A **null pointer** does not point to any valid memory location.
- It is assigned using NULL.

Key Points:

- Initialize pointers to NULL if not used
- Helps avoid **garbage values**
- Always check before dereferencing

Example:

```
int *ptr = NULL;

if(ptr != NULL) {
    printf("%d", *ptr);
}
```

Pointers and Arrays

Array Decay

- Array name becomes a pointer to the first element
- Example:

```
int arr[3] = {1,2,3};
int *ptr = arr;
```

Pointer Arithmetic

- ptr + 1 → moves to next element
- Helps traverse arrays efficiently

Passing Arrays to Functions

- Arrays are passed as pointers

```
void display(int *arr) { }
```

Dynamic Memory Allocation

- Functions:
  - malloc()
  - calloc()
- Used to allocate memory at runtime

Multidimensional Arrays

- Pointers help navigate 2D arrays

```
int arr[2][2];
int (*ptr)[2] = arr;
```

Pointers and Functions

Function Pointers

- Store address of functions

```
void fun() {}
void (*ptr)() = fun;
```

Callback Functions

- Function passed as argument to another function

Array of Function Pointers

- Used for multiple function handling

Polymorphism in C

- Achieved using function pointers

Dynamic Function Calls

- Function can be selected at runtime

Dynamic Memory Allocation

- Allows flexible memory usage during execution

Advantages:

- Efficient memory utilization
- Adapts to changing data size

Important:

- Improper use → **memory leaks**
- Always free memory using:

```
free(ptr);
```

Common Pointer Issues

- Null Pointer Dereferencing
- Memory Leaks
- Wild Pointers (uninitialized pointers)

Best Practices

Initialization and Null Checking

- Always initialize pointers

```
int *ptr = NULL;
```

- Check before use:

```
if(ptr != NULL)
```

Proper Memory Management

- Allocate → Use → Free memory properly

Real-World Applications of Pointers

- Used in:
  - Data Structures (Linked List, Trees)
  - Dynamic memory management
  - System programming
  - File handling
  - Operating systems